

Sécurité implémentée - EcoRide

Date : 11 septembre 2026

Framework : Symfony 8.1.6

PHP : 8.4.25

Vue d'ensemble

Ce document décrit les mécanismes de sécurité mis en place dans l'application EcoRide pour gérer l'authentification, l'autorisation et la suspension de comptes.

1. Authentification des utilisateurs

Configuration

Fichier : config/packages/security.yaml

```
security:
    password_hashers:
        App\Entity\User: 'auto'

    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email

    firewalls:
        main:
            lazy: true
            provider: app_user_provider
            user_checker: App\Security\UserChecker
            form_login:
                login_path: app_login
                check_path: app_login
                enable_csrf: true
                default_target_path: app_home
            logout:
                path: app_logout
```

Fonctionnement

- **Provider** : charge les utilisateurs depuis la table user via leur email.
- **Password hasher** : hachage automatique des mots de passe (algorithme optimal selon Symfony).
- **CSRF** : protection activee contre les attaques par rejeu de formulaire.
- **Logout** : route /logout configurée pour invalider la session.

2. Contrôle de suspension de compte (UserChecker)

Fichier : src/Security/UserChecker.php

```
<?php

namespace App\Security;

use App\Entity\User;
use Symfony\Component\Security\Core\Authentication\Token\TokenInterface;
use Symfony\Component\Security\Core\Exception\CustomUserMessageAccountStatusException;
use Symfony\Component\Security\Core\User\UserCheckerInterface;
use Symfony\Component\Security\Core\User\UserInterface;

class UserChecker implements UserCheckerInterface
{
    public function checkPreAuth(UserInterface $user): void
    {
        if (!$user instanceof User) {
            return;
        }

        if (!$user->isActive()) {
            throw new CustomUserMessageAccountStatusException(
                'Votre compte est suspendu. Contactez l'administrateur.'
            );
        }
    }

    public function checkPostAuth(
        UserInterface $user,
        ?TokenInterface $token = null
    ): void {
        $this->checkPreAuth($user);
    }
}
```

Fonctionnement

- **Avant authentication** (checkPreAuth) : vérifie que `isActive = true`.
- **Après authentication** (checkPostAuth) : vérifie l'état du compte.
- **Exception levée** : `CustomUserMessageAccountStatusException` avec message personnalisé.
- **Résultat** : un utilisateur avec `is_active = 0` ne peut pas se connecter.

3. Entité User

Fichier : `src/Entity/User.php`

Propriétés :

```
#[ORM\Column]
private ?bool $isActive = true;
```

Méthodes :

```
public function isActive(): ?bool
{
    return $this->isActive;
}

public function setIsActive(bool $isActive): static
{
    $this->isActive = $isActive;

    return $this;
}
```

Rôle

- Champ `is_active` dans la table `user` (MySQL).
- Par défaut `true` lors de la création d'un utilisateur.
- Peut être modifié manuellement (phpMyAdmin) ou via une interface admin (venir).

4. Contrôle d'accès par rôles

Configuration : `config/packages/security.yaml`

```
access_control:
    - { path: ^/register, roles: PUBLIC_ACCESS }
    - { path: ^/login, roles: PUBLIC_ACCESS }
    - { path: ^/contact, roles: PUBLIC_ACCESS }
    - { path: ^/carpools, roles: PUBLIC_ACCESS }
```

```

- { path: ^/employee, roles: ROLE_EMPLOYEE }
- { path: ^/admin, roles: ROLE_ADMIN }
- { path: ^/account, roles: ROLE_USER }

```

Rôles implémentés

Rôle	Description	Pages accessibles
PUBLIC_ACCESS	Aucun rôle requis	/register, /login, /contact, /carpools
ROLE_USER	Utilisateur authentifié	/account
ROLE_EMPLOYEE	Employé EcoRide	/employee
ROLE_ADMIN	Administrateur	/admin

Attribution des rôles

Les rôles sont stockés dans la colonne `roles` (JSON) de la table `user`.

Exemple :

```
[ "ROLE_ADMIN", "ROLE_USER" ]
```

5. Fixtures de test

Fichier : `src/DataFixtures/AppFixtures.php`

Utilisateurs créés :

Email	Rôles	isDriver	isPassenger	isActive	Mot de passe
passager1@example.com	ROLE_USER	false	true	true	password123
chauffeur1@example.com	ROLE_USER	true	false	true	password123
mixte@example.com	ROLE_USER	true	true	true	password123
suspendu@example.com	ROLE_USER	true	false	false	password123
employee@ecoride.com	ROLE_EMPLOYEE, ROLE_USER	false	false	true	password123
admin@ecoride.com	ROLE_ADMIN, ROLE_USER	false	false	true	password123

Commande

```
docker compose exec app php bin/console doctrine:fixtures:load --no-interaction
```

6. Tests de validation

Test 1 : Connexion d'un compte actif

```
Email : chauffeur1@example.com  
Mot de passe : password123  
Résultat : ✓ Connexion réussie, redirection vers /
```

Test 2 : Connexion d'un compte suspendu

```
Email : suspendu@example.com  
Mot de passe : password123  
Résultat : ✗ Message "Votre compte est suspendu. Contactez l'administrateur."
```

Test 3 : Accès aux pages protégées

```
URL : /account  
Utilisateur non connecté : → Redirection vers /login  
Utilisateur connecté (ROLE_USER) : ✓ Accès autorisé  
  
URL : /employee  
Utilisateur ROLE_USER : → Accès refusé (403)  
Utilisateur ROLE_EMPLOYEE : ✓ Accès autorisé  
  
URL : /admin  
Utilisateur ROLE_USER : → Accès refusé (403)  
Utilisateur ROLE_ADMIN : ✓ Accès autorisé
```

7. Bonnes pratiques appliquées

- ✓ Hachage automatique des mots de passe (Symfony password_hasher).
- ✓ Protection CSRF sur le formulaire de connexion.
- ✓ Séparation authentification / autorisation.
- ✓ Contrôle d'état du compte avant et après authentification.
- ✓ Rôles granulaires pour l'accès aux pages.
- ✓ Fixtures reproductibles pour les tests.

8. Améliorations prévues

- ☐ Interface admin pour suspendre/réactiver les comptes.
- ☐ Journalisation des tentatives de connexion échouées.
- ☐ Récupération de mot de passe oublié.
- ☐ Validation d'email lors de l'inscription (verify-email-bundle).
- ☐ Rate limiting sur /login (prévention brute-force).

9. Références

- [Documentation Symfony - Security](#)
- [Documentation Symfony - User Checkers](#)
- [Documentation Symfony - Access Control](#)